

Windows API與Code Review

李東霖 博士

國立臺灣海洋大學 電機工程學系

2025 Fall semester

API是什麼？

- API 全名為 **Application Programming Interface**，中文稱為「應用程式介面」，是一種提供不同軟體系統間互動的工具，定義了不同軟體間的互動規範。
- API 允許不同的應用程式、服務或系統之間能夠共享資訊與功能，以約定好的 **API 接口**實現互聯互通。
 - 簡單來說，API 提供了一組定義好的規則與協定，透過資訊的傳遞，讓開發者可以輕鬆整合與使用不同的服務，從而提高應用程式的功能與效能。

使用API的好處

1. 系統整合性高（**Integration**）
 - API 讓不同系統、應用程式或服務之間能夠以標準化的方式交換資料。
 - 開發者可以透過 API 讓內部系統與外部平台（例如支付、地圖、雲端儲存）互相溝通，而不需要了解對方的內部實作。
2. 提升開發效率（**Development Efficiency**）
 - API 提供現成的功能介面，開發者可直接呼叫使用。
 - 減少重複開發的工作量，縮短專案開發週期，並降低維護成本。
3. 模組化與可重用性（**Modularity & Reusability**）
 - API 將系統功能模組化，使不同應用可重複使用同一套服務。
 - 一旦設計出穩定的 API，其他應用程式或團隊可直接使用，維護與擴充更容易。
4. 提升安全性與控制（**Security & Access Control**）
 - API 可透過金鑰（API Key）、OAuth、權限控制等機制保護資料。
 - 能限制哪些應用程式、使用者或服務能存取特定功能或資料，避免系統直接暴露。
5. 便於擴充與創新（**Scalability & Innovation**）
 - API 使企業能快速整合新技術或開放資料給外部開發者。
 - 外部開發者可基於 API 建立創新應用，企業也能更快接軌新服務。

Windows API

- **Windows API**，簡稱法為**WinAPI**，是微軟對於**Windows**作業系統中可用的核心應用程式編程界面的稱法。
- **WinAPI**是用**C**語言所寫的**dll**，可被各種程式語言呼叫，也是套用軟體與**Windows**系統最直接的互動方式。
- **Windows API**提供了8個主要功能：
 1. **基礎服務(Base Services)**: 提供介面，使**Windows**系統裡的基礎資源可以存取。
 2. **進階服務(Advanced Services)**: 提供了對核心以外功能的訪問，包括登錄檔 (**Windows registry**)、關閉/重新啟動系統 (**shutdown/restart**)、服務 (**Windows Service**)、使用者帳戶 (**user accounts**) 等。
 3. **圖形裝置介面(GDI)**: 輸出圖形的內容到外部輸出裝置。
 4. **圖形化使用著介面(GUI)**: 建立和管理螢幕及大多數控制項。
 5. **通用對話方塊連結媒體櫃(Common Dialog Library)**: 是應用程式提供標準的對話方塊，使介面不會太混亂。
 6. **通用控制項連結媒體櫃(Common Control Library)**: 應用程式提供介面，來存取一些作業系統提供的進階控制項。
 7. **Windows外殼(Windows Shell)**: 允許應用程式存取**Windows**外殼所提供的功能。
 8. **網路服務(Network Services)**: 存取作業系統提供的多種網路功能介面。

C#調用Win32 API DLL

1. 引用System.Runtime.InteropServices
 2. 直接從 C# 調用 DLL，使用 C# 關鍵字 static 和 extern 聲明方法。
 - 使用DllImportAttribute 類別來使用API
 - [DllImport("user32.dll")] //替換成所需的DLL檔
 - public static extern ReturnType FunctionName(type arg1, type arg2,...);//替換成所需的方法及參數
- Windows 桌面應用程式的 API 參考
 - <https://learn.microsoft.com/zh-tw/windows/apps/api-reference/>

Windows系統的關機過程與user32.dll

- 系統在接收到使用者發出的關機訊號後(不管是從開始選單的還是按一下電源鍵)，會逐一對所有應用程式發出WM_QUERYENDSESSION訊息，來詢問應用程式在現階段是否可以被關閉
- 如果有一個應用程式對WM_QUERYENDSESSION回傳0，則系統就不會繼續發送WM_QUERYENDSESSION
- 當送完WM_QUERYENDSESSION，那麼系統就會再接著發送WM_ENDSESSION訊息，來「通知」應用程式是否會關機

利用Win32API函式庫user32.dll阻止關機

```
using System.Runtime.InteropServices;
public Form1()
{
    InitializeComponent();
    ShutdownBlockReasonCreate(this.Handle, "不准關不准關");
}
[DllImport("user32.dll")] //調用Win32 API DLL
public extern static bool ShutdownBlockReasonCreate(IntPtr hWnd, [MarshalAs(UnmanagedType.LPWSTR)] string pwszReason);

protected override void WndProc(ref Message aMessage) //取代原有的WndProc功能
{
    const int WM_QUERYENDSESSION = 0x0011; //詢問程序是否可關閉
    const int WM_ENDSESSION = 0x0016; //關閉程序通知

    if (aMessage.Msg == WM_QUERYENDSESSION || aMessage.Msg == WM_ENDSESSION)
        return;

    base.WndProc(ref aMessage); //執行原本內容
}

protected override CreateParams CreateParams //Hide from Task Manager
{
    get
    {
        var cp = base.CreateParams;
        cp.ExStyle |= 0x80;
        return cp;
    }
}
```

C#的多型(Polymorphism)特性實踐 - override

- **override** 的核心作用
 - 重新定義行為：允許子類別根據自己的需求，重新定義繼承自父類別的方法、屬性、索引子或事件的邏輯。
 - 實現多型：這是多型的一種實現方式，讓程式碼能夠在執行階段根據物件的實際類型，呼叫對應的覆寫方法。
- 要使用 **override** 關鍵字，必須滿足以下條件：
 - 父類別的成員必須是可覆寫的：也就是說，該成員在父類別中必須被標記為 **virtual**（虛擬）或 **abstract**（抽象）。
 - 子類別的成員簽章必須完全匹配：覆寫成員的名稱、參數型態、參數數量和回傳值型態必須與父類別中被覆寫的成員完全相同。
 - 不能覆寫非虛擬成員：如果你試圖覆寫一個沒有被標記為 **virtual** 或 **abstract** 的父類別成員，編譯器會報錯。

掛勾(Hook)

- 掛勾是一種機制，應用程式可以利用它攔截事件，例如訊息、滑鼠動作和擊鍵。
- 攔截特定事件類型的函式稱為 攔截程式。掛勾程序可以對它收到的每個事件進行處理，然後修改或捨棄該事件。
- 掛鈎的一些用途：
 - 監視訊息以進行偵錯
 - 提供巨集錄製和播放支援
 - 提供 F1 說明鍵的支援
 - 模擬滑鼠和鍵盤輸入
 - 實作電腦型訓練（CBT）應用程式

掛勾類型

- 系統支援多種不同類型的掛勾；每種類型提供對其訊息處理機制不同層面的存取。
 - **WH_CALLWNDPROC** 和 **WH_CALLWNDPROCRET**: 監控傳送至視窗程序的訊息
 - **WH_CBT**: 決定系統是否允許或防止其中一項作業:
 - 系統在啟動、建立、終結、最小化、最大化、移動或調整視窗大小之前
 - 在完成系統命令之前
 - 在從系統訊息隊列中移除滑鼠或鍵盤事件之前；在設定輸入焦點之前
 - 與系統訊息隊列同步處理之前
 - **WH_DEBUG**: 在系統使用其他Hook之前，提供除錯功能
 - **WH_FOREGROUNDIDLE**: 監控程序傳出的訊息: 攔截可讓您在前景線程閒置時執行低優先順序的工作
 - **WH_GETMESSAGE**: 監視即將由 **GetMessage** 或 **PeekMessage** 函式傳回的訊息
 - **WH_JOURNALPLAYBACK** (Win11後不再支援): 允許應用程式將訊息插入到系統訊息佇列中
 - **WH_JOURNALRECORD** (Win11後不再支援): 用來監視和記錄輸入事件
 - **WH_KEYBOARD_LL**: 監視全系統的鍵盤訊息
 - **WH_KEYBOARD**: 監視指定視窗的鍵盤訊息
 - **WH_MOUSE_LL**: 監視全系統的滑鼠訊息
 - **WH_MOUSE**: 監視指定視窗的滑鼠訊息
 - **WH_MSGFILTER** 和 **WH_SYSMSGFILTER**: 監視功能表、滾動條、訊息框或對話框即將處理的訊息
 - **WH_SHELL**

掛勾程序SetWindowsHookEx

- SetWindowsHookEx是Windows作業系統中用來安裝訊息掛鈎的API函數，屬於視窗訊息處理機制的核心元件。
- 此函數透過設定掛鈎程序截取指定類型的訊息，允許在訊息到達目標視窗前進行預處理(包含本程序或其他程序的視窗訊息)。
- SetWindowsHookEx函數包含四個參數：
 - idHook指定掛鈎類型（參考上頁）
 - lpfn指向回呼函數位址，
 - hMod為包含掛鈎程式碼的模組句柄，
 - dwThreadId決定關聯執行緒
- 安裝後的掛鈎程序位於掛鈎鏈的開頭，系統透過CallNextHookEx在鍊錶中傳遞訊息。

全局監控鍵盤範例(1) - 建立Hook (1)

```
class KeyboardHook
{
    public event KeyEventHandler KeyDownEvent;
    public event KeyPressEventHandler KeyPressEvent;
    public event KeyEventHandler KeyUpEvent;

    public delegate int HookProc(int nCode, Int32 wParam, IntPtr lParam);
    static int hKeyboardHook = 0; //宣告鍵盤掛勾的初始值
    //對應數值可在Microsoft SDK的Winuser.h里查詢

    public const int WH_KEYBOARD_LL = 13; //單程序監控投為2，全系統監控設為13
    HookProc KeyboardHookProcedure; //宣告KeyboardHookProcedure為HookProc類型
    //鍵盤結構
    [StructLayout(LayoutKind.Sequential)]
    public class KeyboardHookStruct
    {
        public int vkCode; //虛擬鍵碼。範圍1至254
        public int scanCode; // 硬體掃描碼
        public int flags; // 旗標
        public int time; // 時間戳記
        public int dwExtraInfo; // 額外資訊
    }
}
```

全局監控鍵盤範例(2) - 建立Hook (2)

//用來安裝掛勾

```
[DllImport("user32.dll", CharSet = CharSet.Auto, CallingConvention = CallingConvention.StdCall)]  
public static extern int SetWindowsHookEx(int idHook, HookProc lpfn, IntPtr hInstance, int threadId);
```

//用來卸載掛勾

```
[DllImport("user32.dll", CharSet = CharSet.Auto, CallingConvention = CallingConvention.StdCall)]  
public static extern bool UnhookWindowsHookEx(int idHook);
```

//用來呼叫下一個掛鉤

```
[DllImport("user32.dll", CharSet = CharSet.Auto, CallingConvention = CallingConvention.StdCall)]  
public static extern int CallNextHookEx(int idHook, int nCode, IntPtr wParam, IntPtr lParam);
```

// 取得當前程序編號（如採單程序監控投需要）

```
[DllImport("kernel32.dll")]  
static extern int GetCurrentThreadId();
```

//使用WINDOWS API函數取代取得目前執行個體的函數,防止掛鉤失效

```
[DllImport("kernel32.dll")]  
public static extern IntPtr GetModuleHandle(string name);
```

全局監控鍵盤範例(3) - 建立Hook (3)

```
public void Start()
{
    // 安裝鍵盤掛鉤
    if (hKeyboardHook == 0)
    {
        KeyboardHookProcedure = new HookProc(KeyboardHookProc);
        hKeyboardHook = SetWindowsHookEx(WH_KEYBOARD_LL, KeyboardHookProcedure,
            GetModuleHandle(System.Diagnostics.Process.GetCurrentProcess().MainModule.ModuleName), 0);

        //如果SetWindowsHookEx失敗
        if (hKeyboardHook == 0)
        {
            Stop();
            throw new Exception("安裝鍵盤掛鉤失敗");
        }
    }
}

public void Stop()
{
    bool retKeyboard = true;
    if (hKeyboardHook != 0)
    {
        retKeyboard = UnhookWindowsHookEx(hKeyboardHook);
        hKeyboardHook = 0;
    }
    if (!(retKeyboard)) throw new Exception("卸載掛鉤失敗!");
}
```

全局監控鍵盤範例(4) - 建立Hook (4)

//將指定的虛擬鍵碼和鍵盤狀態的轉換成對應字元

[DllImport("user32")]

public static extern int ToAscii(int uVirtKey, //[in] 虛擬關鍵程式碼
int uScanCode, // [in] 硬體掃描碼
byte[] lpbKeyState, // [in] 256位元組指針，包含目前鍵盤的狀態。
byte[] lpwTransKey, // [out] 指標的緩衝區：用來接收字元。
int fuState); // [in] 活動狀態

//取得按鍵的狀態

[DllImport("user32")]

public static extern int GetKeyboardState(byte[] pbKeyState);

[DllImport("user32.dll", CharSet = CharSet.Auto, CallingConvention = CallingConvention.StdCall)]

private static extern short GetKeyState(int vKey);

private const int WM_KEYDOWN = 0x100; //KEYDOWN

private const int WM_KEYUP = 0x101; //KEYUP

private const int WM_SYSKEYDOWN = 0x104; //SYSKEYDOWN

private const int WM_SYSKEYUP = 0x105; //SYSKEYUP

全局監控鍵盤範例(5) - 建立Hook (5)

```
private int KeyboardHookProc(int nCode, Int32 wParam, IntPtr lParam)
{
    // 偵聽鍵盤事件
    if ((nCode >= 0) && (KeyDownEvent != null || KeyUpEvent != null || KeyPressEvent != null))
    {
        KeyboardHookStruct MyKeyboardHookStruct = (KeyboardHookStruct)Marshal.PtrToStructure(lParam, typeof(KeyboardHookStruct));
        // raise KeyDown
        if (KeyDownEvent != null && (wParam == WM_KEYDOWN || wParam == WM_SYSKEYDOWN))
        {
            Keys keyData = (Keys)MyKeyboardHookStruct.vkCode;
            KeyEventArgs e = new KeyEventArgs(keyData);
            KeyDownEvent(this, e);
        }

        //鍵盤按下
        if (KeyPressEvent != null && wParam == WM_KEYDOWN)
        {
            byte[] keyState = new byte[256];
            GetKeyboardState(keyState);

            byte[] inBuffer = new byte[2];
            if (ToAscii(MyKeyboardHookStruct.vkCode, MyKeyboardHookStruct.scanCode, keyState, inBuffer, MyKeyboardHookStruct.flags) == 1)
            {
                KeyPressEventArgs e = new KeyPressEventArgs((char)inBuffer[0]);
                KeyPressEvent(this, e);
            }
        }
    }
}
```


全局監控鍵盤範例(6) - 建立Hook (6)

```
// 鍵盤放開
if (KeyUpEvent != null && (wParam == WM_KEYUP || wParam == WM_SYSKEYUP))
{
    Keys keyData = (Keys)MyKeyboardHookStruct.vkCode;
    KeyEventArgs e = new KeyEventArgs(keyData);
    KeyUpEvent(this, e);
}

//如果回傳1，則結束訊息，這個訊息到此為止，不再傳遞。
//如果回傳0或呼叫CallNextHookEx函數則訊息出了這個掛鉤繼續往下傳遞，也就是傳給訊息真正的接受者
return CallNextHookEx(hKeyboardHook, nCode, wParam, lParam);
}
~KeyboardHook()
{
    Stop();
}
}
```

全局監控鍵盤範例(7) – 使用掛勾

```
private void Form1_Load(object sender, EventArgs e)
{
    //安裝鍵盤掛鉤
    var k_hook = new KeyboardHook();
    k_hook.KeyDownEvent += new KeyEventHandler(hook_KeyDown);
    k_hook.Start();
}

private void hook_KeyDown(object sender, KeyEventArgs e)
{
    //判斷是否按下alt + a
    if (e.KeyValue == (int)Keys.A && (int)Control.ModifierKeys == (int)Keys.Alt)
        //if (e.KeyValue == (int)Keys.A )
        {
            MessageBox.Show("按下了Alt + A");
        }
}
```

Code Review on GitHub

Code Review是什麼？

- Code Review 是一種軟體開發過程中的重要步驟，指的是團隊成員對程式碼進行檢查與審核的過程。當開發者提交程式碼修改（例如 Pull Request 或 Merge Request）時，其他成員會檢視這些修改，確保程式碼的正確性、可讀性、安全性以及品質。
- 透過回饋使團隊整體程式品質更好
- 不是攻擊個人、強迫對方寫出自己想要的程式

審查前後的比較

審查前

```
#include <stdio.h>

char conv(int x){
    char r;
    if(x>=90)
        r = 'A';
    else if(x>=80)
        r = 'B';
    else if(x>=70)
        r = 'C';
    else if(x>=60)
        r = 'D';
    else
        r = 'F';
    return r;
}

int main()
{
    int x;
    printf("Enter score: ");
    scanf("%d", &x);
    printf("%c\n", conv(x));
    return 0;
}
```



審查後

```
#include <stdio.h>

char convert_score_to_grade(int score) {
    // 根據成績回傳對應的等第
    if (score >= 90)
        return 'A';
    else if (score >= 80)
        return 'B';
    else if (score >= 70)
        return 'C';
    else if (score >= 60)
        return 'D';
    else
        return 'F';
}

int main() {
    int score;
    char grade;

    printf("請輸入成績 (0-100): ");
    if (scanf("%d", &score) != 1) {
        printf("請輸入整數!\n");
        return 1; // 結束程式
    }
    if (score < 0 || score > 100) {
        printf("成績必須在 0 到 100 之間。 \n");
        return 1;
    }
    grade = convert_score_to_grade(score);
    printf("等第為 : %c\n", grade);

    return 0;
}
```

Code Review的目的

- 錯誤預防
 - 在整合程式碼變更前就能發現潛在錯誤與bug
- 知識共享
 - 開發成員可透過Review 學習他人寫法與設計模式
- 維護一致性
 - 統一團隊的風格與架構，提升可讀性與維護性
- 增進溝通
 - 促進團隊技術溝通與共識形成

Code Review的角色與分工

1. 作者(Author)

- 撰寫程式碼、提交Review 請求

2. 審查者(Reviewer)

- 審查程式碼、提出回饋意見

3. 維護者(Maintainer)

- 合併分支、最終批准者

Code Review的流程



Code Review 態度與原則

- 聚焦在程式碼非個人
 - 對事不對人，例如：變數命名不清楚，除非你不會寫code
- 客觀依據標準
 - 遵循團隊風格指南，避免主觀爭論
- 提供具體建議
 - 不只指出問題，也建議怎麼修改
- 使用正面語氣
 - 建設性語言，例如：「也許可以考慮這樣寫會更清楚」
- 小步前進
 - 不要求一次改完全部，強調漸進式改善

Code Review要審查什麼？

- 程式碼品質（ Code Quality ）
- 程式碼風格（ Code Style ）
- 邏輯正確性（ Logic Correctness ）
- 安全性與穩定性（ Security & Robustness ）

程式碼品質

- 程式碼在可讀性、可維護性、可測試性、可重用性、效率與正確性等方面的整體品質水準

品質好的程式碼應該：

1. 容易被他人理解
2. 容易修改與擴充
3. 不容易出錯，且出錯時容易偵錯
4. 符合軟體工程的最佳實務

不好的程式碼品質：

1. 命名不清: `a, b, c, x, y, z` 等無意義變數
2. 函式太長: 整個功能寫在一個函式內，難以閱讀與重用
3. 重複邏輯: 同樣的處理流程出現多次，沒抽成函式
4. 巢狀太深: `if → for → if → while.....` 讓邏輯難以追蹤
5. 缺乏註解: 其他人無法快速理解你的設計意圖

審查程式碼品質

- 是否可讀性高
- 是否有重複程式碼
- 是否有適當的模組化與函式劃分
- 是否有註解與文件

審查程式碼品質範例

```
#include <stdio.h>

double ct(double p, double q, double t)
{
    double tax, total;
    if (t) {
        tax = p * q * 0.05;
    } else {
        tax = 0;
    }
    total = p * q + tax;
    printf("Total amount: %.2f\n", total);
    return total;
}
```



- 可讀性（變數命名、縮排）
- 可維護性（函式重複使用）
- 測試性（是否可unit test）

```
#include <stdio.h>
#include <stdbool.h>
// 根據價格與數量計算總價，若需加稅則加上5%稅金
// param price: 單價
// param quantity: 數量
// param taxable: 是否需加稅 (true/false)
// return: 總價 (含稅)
double calculate_total_price(double price, int quantity, bool taxable) {
    double subtotal = price * quantity;
    double tax = taxable ? subtotal * 0.05 : 0.0;
    return subtotal + tax;
}

int main() {
    double price, total;
    int quantity;
    char tax_input;
    bool taxable;

    printf("請輸入單價：");
    if (scanf("%lf", &price) != 1) {
        printf("輸入錯誤！請輸入數值。\\n");
        return 1;
    }
    printf("請輸入數量：");
    if (scanf("%d", &quantity) != 1) {
        printf("輸入錯誤！請輸入整數。\\n");
        return 1;
    }
    printf("是否需加稅？(Y/N)：");
    scanf(" %c", &tax_input);
    taxable = (tax_input == 'Y' || tax_input == 'y');
    total = calculate_total_price(price, quantity, taxable);
    printf("Total is: $%.2f\\n", total);
    return 0;
}
```

程式碼風格

- 程式碼在排版、命名、縮排、空格、註解、程式結構等規範的撰寫規範與慣例
- 程式碼風格不影響程式正確性，但會大大影響可讀性、維護性、團隊合作效率與專業度

為什麼要遵守程式碼風格？

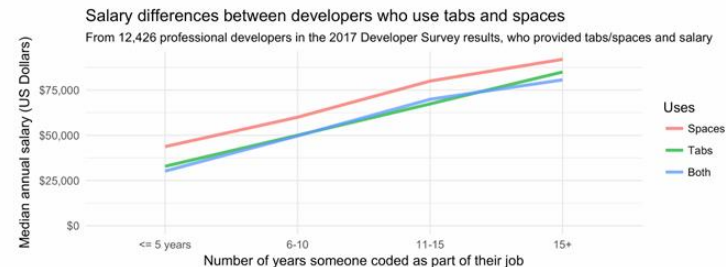
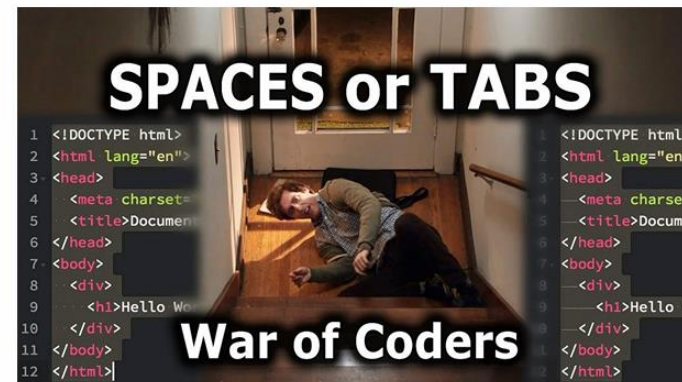
- 提高可讀性
 - 讓別人（或自己）能快速理解程式內容
- 團隊一致性
 - 讓多人開發的程式碼風格統一、整齊
- 降低錯誤率
 - 清晰結構有助於發現潛在問題
- 容易維護與測試
 - 格式統一便於除錯與版本追蹤

常見的程式碼風格標準

- C
 - K&R Style
 - GNU Coding Standards
- C++
 - Google C++ Style Guide
 - LLVM Coding Standards
- C#
 - Microsoft C# Style Guide
 - .NET Coding Conventions
- Python
 - PEP 8 (Python Enhancement Proposal #8)
- Java
 - Google Java Style Guide
 - Oracle Conventions
- JavaScript
 - Airbnb Style Guide
 - StandardJS

審查程式碼風格

1. 是否遵循標準風格規範
 - getAmount vs get_amount
 - tab vs space
2. 命名是否一致、有意義（函式/變數/常數）
3. 縮排、間距、空行等是否整齊



審查程式碼風格範例

```
#include <stdio.h>

void calArea(double I, double b){
double area=I*b;
printf("area is %.2f\n", area);
}
```

- 命名不清楚
- 縮排不正確
- 空格使用不同
- 字串處理風格差
- 無docstring



```
#include <stdio.h>
// 計算矩形面積
// param length: 長度
// param breadth: 寬度
// return: 面積
double calculate_area(double length, double breadth) {
    double area = length * breadth;
    printf("Area is %.2f\n", area);
    return area;
}
int main() {
    double length, breadth, area;

    printf("請輸入長度：");
    if (scanf("%lf", &length) != 1) {
        printf("輸入錯誤！請輸入數值。\\n");
        return 1;
    }
    printf("請輸入寬度：");
    if (scanf("%lf", &breadth) != 1) {
        printf("輸入錯誤！請輸入數值。\\n");
        return 1;
    }
    area = calculate_area(length, breadth);
    printf("矩形面積為：%.2f\\n", area);
    return 0;
}
```

邏輯正確性

- 程式執行的流程是否符合預期的運算邏輯，能正確解決問題並產生正確的結果
- 一段邏輯正確的程式碼：
 1. 條件判斷正確無誤
 2. 流程控制合理（**if/else**、迴圈、函式）
 3. 能正確處理所有預期輸入
 4. 在各種邊界或例外情況下也能產生正確行為

為什麼邏輯正確性重要？

- 正確性
 - 程式若邏輯錯誤，即使沒有語法錯誤也可能產生錯誤結果(語義錯誤)
- 難以偵錯
 - 邏輯錯誤通常不會立即報錯，但會導致不正確輸出
- 使用者信任
 - 錯誤的商業邏輯或資料處理可能導致使用者流失或資料毀損

審查邏輯正確性

1. 條件判斷

- 是否考慮所有可能輸入與邊界條件？是否有順序錯誤或重疊？

2. 迴圈邏輯

- 是否會無限迴圈？是否有正確的終止條件？

3. 函式流程

- 是否回傳正確結果？是否傳遞正確的參數與資料？

4. 特殊情況

- 輸入為0、空字串、負數時是否正確處理

審查邏輯正確性範例(1)

- 條件判斷邏輯錯誤

```
void check_age(int age)
{
    if (age > 18) {
        printf("成年\n");
    } else if (age > 12) {
        printf("青少年\n");
    } else if (age > 0) {
        printf("兒童\n");
    } else {
        printf("無效年齡\n");
    }
}
```



```
void check_age(int age)
{
    if (age > 18) {
        printf("成年\n");
    } else if (age > 13) {
        printf("青少年\n");
    } else if (age > 0) {
        printf("兒童\n");
    } else {
        printf("無效年齡\n");
    }
}
```

審查邏輯正確性範例(2)

- 流程控制邏輯錯誤 – 印出到10之間的偶數

```
#include <stdio.h>

void print_even_numbers()
{
    int i = 1;
    while (i <= 10)
    {
        if (i % 2 == 0)
            printf("%d\n", i);
        else
            i++;
    }
}
```



```
#include <stdio.h>

void print_even_numbers()
{
    int i = 1;
    while (i <= 10)
    {
        if (i % 2 == 0)
        {
            printf("%d\n", i);
        }
        i++;
    }
}
```

審查邏輯正確性範例(3)

- 預期輸入錯誤 – 計算使用者輸入數字的平方根

```
#include <stdio.h>
#include <math.h>

void compute_square_root()
{
    double number, result;
    printf("輸入一個數值:\n");
    scanf("%lf", &number);
    result = sqrt(number);
    printf("平方均為 %.2f\n", result);
}
```



- 輸入非數字
- 輸入負數
- 缺少提示

```
#include <stdio.h>
#include <math.h>

void compute_square_root()
{
    double number, result;
    printf("輸入一個非負數值:\n");
    if (scanf("%lf", &number) != 1) {
        printf("無效輸入，請輸入數值。\\n");
        return;
    }
    if (number < 0) {
        printf("無效輸入，請輸入非負數值。\\n");
        return;
    }
    result = sqrt(number);
    printf("平方均為 %.2f\n", result);
}
```


審查邏輯正確性範例(4)

檢查函式是否接收到正確資料

```
#include <stdio.h>
int check_pass(float grade) {
    if (grade >= 60) {
        return 1;
    } else {
        return 0;
    }
}
int main() {
    float grade;
    printf("輸入你的成績: ");
    scanf("%f", &grade);

    if (check_pass(grade)) {
        printf("及格\n");
    } else {
        printf("不及格\n");
    }
    return 0;
}
```



- 未確保輸入是數值
- 無錯誤處理

```
#include <stdio.h>
// 函式用來檢查成績是否及格
int check_pass(float grade) {
    if (grade >= 60) {
        return 1;
    } else {
        return 0;
    }
}
int main() {
    float grade;
    printf("輸入你的成績: ");
    while (scanf("%f", &grade) != 1) {
        printf("無效輸入，請輸入數值。\\n");
        while (getchar() != '\\n'); // 清除輸入緩衝區
        printf("請重新輸入你的成績: ");
    }
    if (check_pass(grade)) {
        printf("及格\\n");
    } else {
        printf("不及格\\n");
    }
    return 0;
}
```

安全性與穩定性

- 安全性
- 指程式在運作過程中，是否能防範外部攻擊、資料洩漏與惡意輸入等風險，確保使用者資料與系統資源的安全
- 穩定性
 - 指程式在面對非預期輸入、錯誤操作、邊界條件、異常情境時，是否仍能穩定運行、不當掉或產生錯誤結果

為什麼安全性與穩定性重要？

- 安全性
 - 防止SQL injection、XSS、密碼洩漏、權限濫用等
- 穩定性
 - 避免崩潰、無窮迴圈、記憶體爆炸、錯誤結果或資料毀損
- 對用戶體驗與信任事關重要
 - 不安全或不穩定的系統會造成用戶流失與損失

審查安全性與穩定性

1. 安全性

- 輸入驗證、敏感資料保護、安全 API、權限控管、記憶體安全

2. 穩定性

- 錯誤處理、邊界檢查、資源釋放、異常回復、執行緒安全

3. 可維護性

- 命名清晰、模組化、註解適當、測試覆蓋充分

4. 相依與部署

- 外部套件版本安全、配置合理、環境變數管理正確

審查安全性與穩定性範例

陣列越界(Index Out of Range)

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int index;

    printf("請輸入要存取的索引值 (0~4) : ");
    scanf("%d", &index);
    printf("arr[%d] = %d\n", index, arr[index]);

    return 0;
}
```



```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int index;

    printf("請輸入要存取的索引值 (0~4) : ");
    if (scanf("%d", &index) != 1) {
        printf("輸入錯誤！請輸入整數。\\n");
        return 1;
    }

    // 檢查是否越界
    if (index < 0 || index >= 5) {
        printf("錯誤：索引 %d 超出陣列範圍！\\n", index);
    } else {
        printf("arr[%d] = %d\\n", index, arr[index]);
    }

    return 0;
}
```

- 未確保存取索引值在範圍內

撰寫有意義的審查評論

- 好的評論不僅指出問題，更應該：
 1. 具體: 指出是哪一行或哪個函式
 - e.g. line 12: 此條件可能漏掉`score = 60`的情況
 2. 合理性: 說明影響或風險
 - e.g. 這裡未處理輸入非數字的例外，會導致程式崩潰
 3. 建設性: 提出可行改善方案
 - e.g. 建議使用`try-except` 包裝輸入轉換，處理`ValueError`
 4. 尊重對方: 語氣中立、不攻擊
 - e.g. 這裡的寫法清楚，但或許可以用更簡化的方式來提升可讀性

撰寫有意義的審查評論範例(1)

- 針對輸入錯誤未處理的評論

```
score = int(input("請輸入成績："))
```

- 審查評論：

建議補上例外處理機制，例如使用**try-except**，因為如果使用者輸入非數字（例如**abc**），這行會產生**ValueError**並導致程式終止。為了提升程式穩定性，可以考慮提示錯誤並要求重新輸入

撰寫有意義的審查評論範例(2)

- 針對命名不清楚的評論

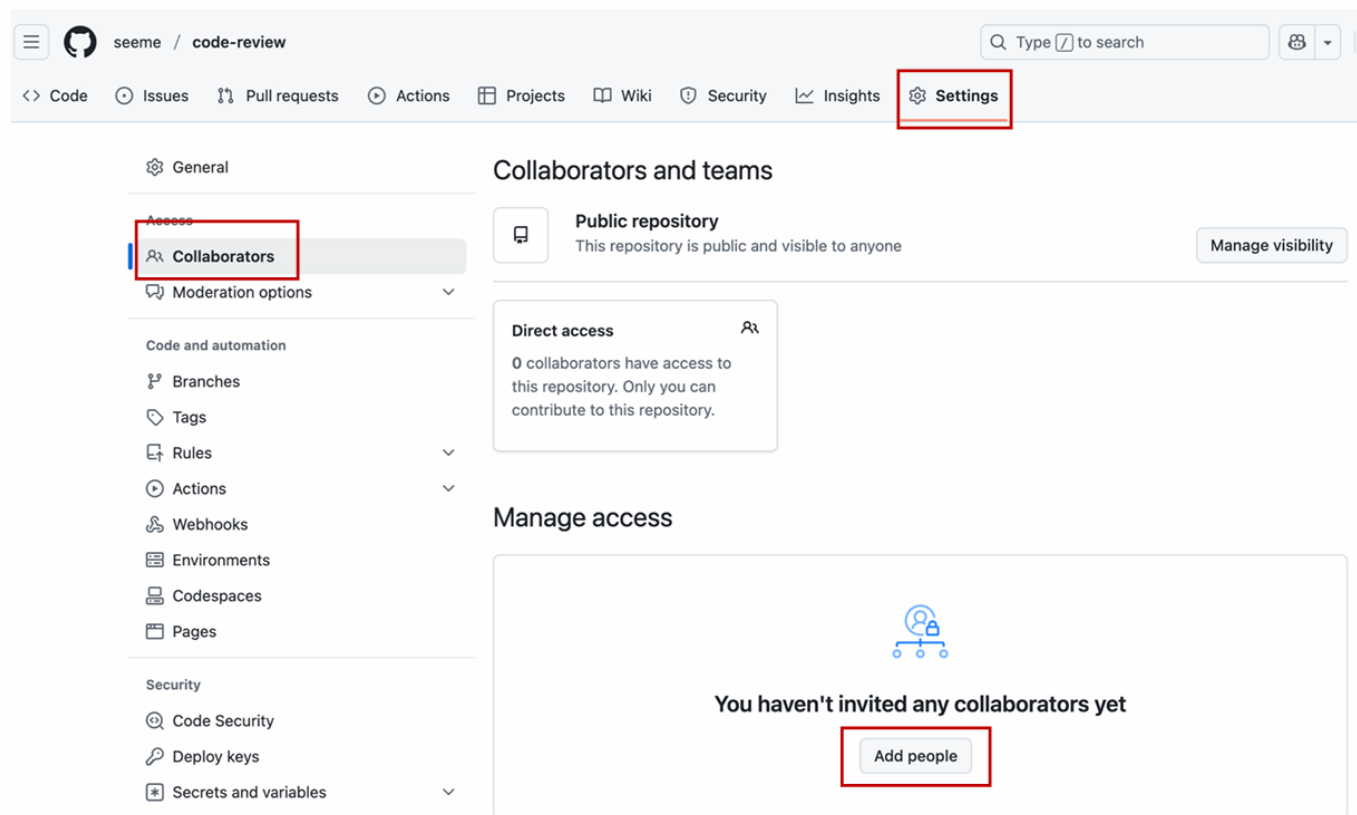
```
int ct(int p, int q){  
    return p * q;  
}
```

- 審查評論：

建議將`ct`, `p`, `q` 命名為更具意義的名稱，例如`calculate_total(price, quantity)`。
這樣可以讓閱讀程式的同學或未來的維護者更快理解函式用途，提升可讀性與可維護性。

Review C# Code

新增協同開發者(1)



新增協同開發者(2)

The screenshot shows the GitHub 'Collaborators and teams' settings page for a public repository. A modal titled 'Add people to code-review' is open, prompting a search by username, full name, or email. The search input field contains 'fcu-d1234567' and is highlighted with a red box. Below the search bar, a search result for 'HSI-MIN CHEN' with the username 'fcu-d1234567' and the role 'Invite collaborator' is displayed. At the bottom of the modal, there are two buttons: 'Cancel' and 'Add to repository', with the latter highlighted by a red box. The background interface includes a sidebar with navigation options like 'General', 'Access', 'Collaborators', 'Moderation options', 'Code and automation', 'Branches', 'Tags', 'Rules', 'Actions', 'Webhooks', 'Environments', 'Codespaces', and 'Pages'. The main header area shows 'Public repository' status and a 'Manage visibility' button.

General

Collaborators and teams

Public repository
This repository is public and visible to anyone

Manage visibility

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Add people to code-review

Search by username, full name, or email

fcu-d1234567

HSI-MIN CHEN
fcu-d1234567 • Invite collaborator

Cancel Add to repository

新增協同開發者(3)

The screenshot displays the GitHub repository settings interface. On the left is a sidebar with navigation links: General, Access, Collaborators (highlighted), Moderation options, Code and automation (with sub-links for Branches, Tags, Rules, Actions, Webhooks, Environments, Codespaces, and Pages), and Security (with sub-links for Code Security, Deploy keys, and Secrets and variables). The main content area is titled 'Collaborators and teams'. It features a 'Public repository' status box with a 'Manage visibility' button. Below this is a 'Direct access' box indicating that one user has access, with links to view collaborators and invitations. The 'Manage access' section includes an 'Add people' button, a 'Select all' checkbox, a 'Type' dropdown, and a search bar labeled 'Find a collaborator...'. A table lists the current collaborator, HSI-MIN CHEN, with a status of 'Awaiting fcu-d1234567's response' and a 'Pending Invite' button. The entire 'Manage access' section is enclosed in a red rectangular border.

Collaborators and teams

Public repository
This repository is public and visible to anyone [Manage visibility](#)

Direct access
1 user has access to this repository. [0 collaborators](#), [1 invitation](#).

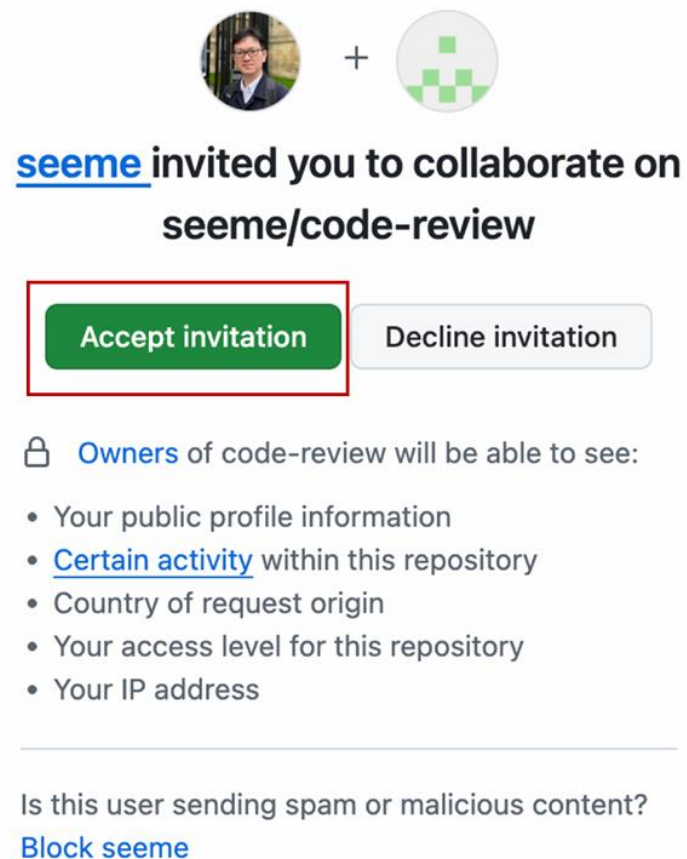
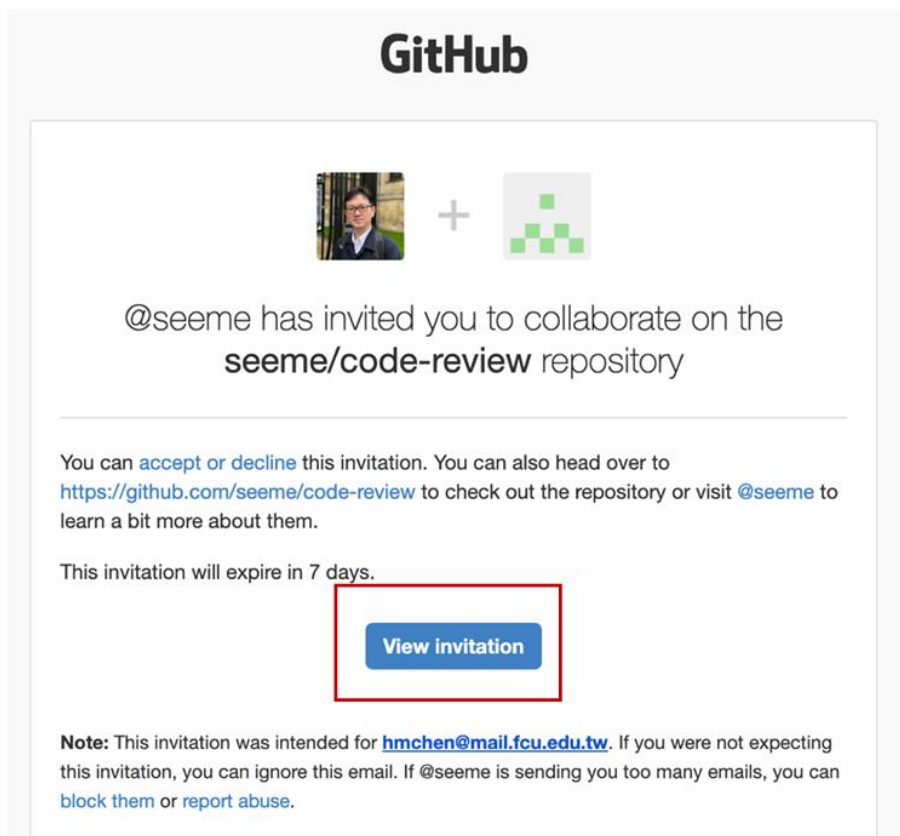
Manage access [Add people](#)

☐ Select all Type ▾

Find a collaborator...

☐	 HSI-MIN CHEN Awaiting fcu-d1234567's response	Pending Invite 	
--------------------------	---	--	---

新增協同開發者(4)



審查程式碼(1)

The screenshot shows the GitHub interface for the repository 'seeme / code-review'. The 'Pull requests' tab is highlighted with a red box and shows a count of 1. Below the repository name, there are buttons for 'Pin', 'Unwatch', and a count of 1. The main content area shows a commit by 'seeme' with the message 'Initial commit', hash 'cc903a6', and time '10 minutes ago'. A file 'README.md' is listed with the same commit information. Below this, the 'README' section is visible, containing the text 'code-review'.

seeme / code-review

Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

code-review Public

main 2 Branches 0 Tags

Go to file Add file Code

seeme Initial commit cc903a6 · 10 minutes ago 1 Commit

README.md Initial commit 10 minutes ago

README

code-review

審查程式碼(2)

The screenshot shows a GitHub pull request page for a repository named '新增功能-1 #1'. The top navigation bar includes links for Code, Issues, Pull requests (1), Actions, Projects, Wiki, Security, Insights, and Settings. The pull request title is '新增功能-1 #1'. Below the title, it states 'fcu-d1234567 wants to merge 1 commit into main from feature-1'. A red box highlights the 'Files changed 1' tab in the navigation bar. The main content area shows a comment from 'fcu-d1234567' (a collaborator) made 3 minutes ago, with the text '開發面積計算函式'. Below the comment, a commit is shown: '新增test.py' by 'fcu-d1234567' assigned to 'seeme' 3 minutes ago. At the bottom, a green box indicates 'No conflicts with base branch' and 'Merging can be performed automatically.' with a 'Merge pull request' button and a link to 'View command line instructions'.

<> Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

新增功能-1 #1

Open fcu-d1234567 wants to merge 1 commit into main from feature-1

Conversation 0 Commits 1 Checks 0 Files changed 1

fcu-d1234567 commented 3 minutes ago Collaborator ...

開發面積計算函式

新增test.py f3031cd

fcu-d1234567 assigned seeme 3 minutes ago

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

審查程式碼(3)

新增功能-1 #1

 Open fcu-d1234567 wants to merge 1 commit into `main` from `feature-1` 

 Conversation 0  Commits 1  Checks 0  Files changed 1

Changes from all commits  File filter  Conversations  Jump to  

3  test.py 

```
... @@ -0,0 +1,3 @@  
1 + def calArea(l,b):  
2 +   area=l*b  
3 +   print("area is "+str(area))  
  -
```

 © 2025 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#)

Edit  Code 

+3 -0 

Finish your review

 Review changes 

Write Preview               

建議修正下列程式碼品質問題

- 命名不清楚
- 縮排不正確
- 空格使用不當



 Markdown is supported

 Paste, drop, or click to add files

☒ Comment

Submit general feedback without explicit approval.

☐ Approve

Submit feedback and approve merging these changes.

☐ Request changes

Submit feedback that must be addressed before merging.

Submit review

審查程式碼(4)

新增功能-1 #1

Edit <> Code

Open fcu-d1234567 wants to merge 1 commit into main from feature-1

Conversation 1 Commits 1 Checks 0 Files changed 1

+3 -0



fcu-d1234567 commented 11 minutes ago

Collaborator

開發面積計算函式



新增test.py

f3031cd

fcu-d1234567 assigned seeme 11 minutes ago



seeme reviewed now

View reviewed changes

seeme left a comment

Owner

建議修正下列程式碼品質問題

- 命名不清楚
- 縮排不正確
- 空格使用不當



Reviewers

seeme



Still in progress? [Convert to draft](#)

Assignees

seeme

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

審查程式碼(5)

Re: [seeme/code-review] 新增功能-1 (PR #1) 外部 收件匣 x



Hsi-Min Chen <notifications@github.com>

[取消訂閱](#)

寄給 seeme/code-review、我、Author ▼



[翻譯成中文（繁體）](#)



@seeme commented on this pull request.

建議修正下列程式碼品質問題

- 命名不清楚
- 縮排不正確
- 空格使用不當

—

Reply to this email directly, [view it on GitHub](#), or [unsubscribe](#).

You are receiving this because you authored the thread.